

## Assignment No. 12

### Spatial and Geographic Data Geospatial is the natural domain for Graph Database

#### Use Neo4j and Neo4j Spatial

PRN: 23520008 Batch: T7

Problem Statement: Finding Things Close to Other Things.

Application in: location-based services on the web

#### **Aim**

The aim of this assignment is to explore and implement geospatial queries using Neo4j graph database along with Neo4j Spatial plugin. Specifically, the task is to add randomly generated location points to the database and use Cypher Query Language (CQL) to query the nearest locations to one another using the point() and distance() functions. The primary objective is to demonstrate the use of Neo4j for location-based services and spatial operations.

#### **Objectives**

1. Install and configure Neo4j Spatial plugin to facilitate geospatial operations.
2. Generate 10,000 random location points and add them into the Neo4j database.
3. Write and execute Cypher queries to find the closest or nearest things based on geospatial proximity.
4. Use Neo4j's spatial functions like point() and distance() for spatial queries.
5. Demonstrate how location-based queries can be implemented for practical applications like geospatial searches in web services.

#### **Algorithm**

##### **1. Install and Configure Neo4j Spatial Plugin:**

- Download and install the Neo4j Spatial plugin from [GitHub](#).
- Follow the instructions to integrate it into your Neo4j installation.
- Verify the plugin by checking if spatial functions like distance() and point() are available.

##### **2. Generate and Add 10,000 Random Location Points:**

- Define random coordinates for each of the 10,000 location points (latitude, longitude).
- Create Place nodes in Neo4j with a property for each location's coordinates.

### **3. Execute and Validate Queries:**

- Test the queries by running them in the Neo4j browser and verify that the results return the correct nearest locations.
- Ensure that the plugin's geospatial functions are correctly interpreting the data and calculating distances between locations.

## **Procedure**

### **1. Install Neo4j and Neo4j Spatial Plugin:**

- Download and install Neo4j from the official website.
- Download the Neo4j Spatial plugin from GitHub and configure it by placing the plugin in the plugins/ directory of Neo4j.
- Restart Neo4j to load the plugin.

### **2. Create Random Location Data:**

- Use a scripting language like Python or JavaScript to generate 10,000 random geographic coordinates (latitude and longitude).
- Write a script to add these points as nodes into Neo4j with a Place label and a location property for each point's coordinates.

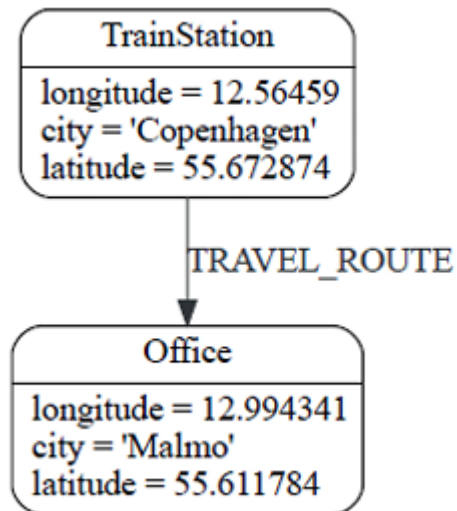
### **3. Write and Execute Cypher Queries:**

- Use the Cypher queries mentioned earlier to find the nearest places to a specified location or another place.
- Run queries in the Neo4j browser or through a Neo4j client (e.g., using Python's neo4j package) to fetch and display results.

### **4. Optimize and Analyze:**

- Test the scalability and performance of the queries, especially when handling a large number of locations.
- Consider indexing location data for faster search performance.

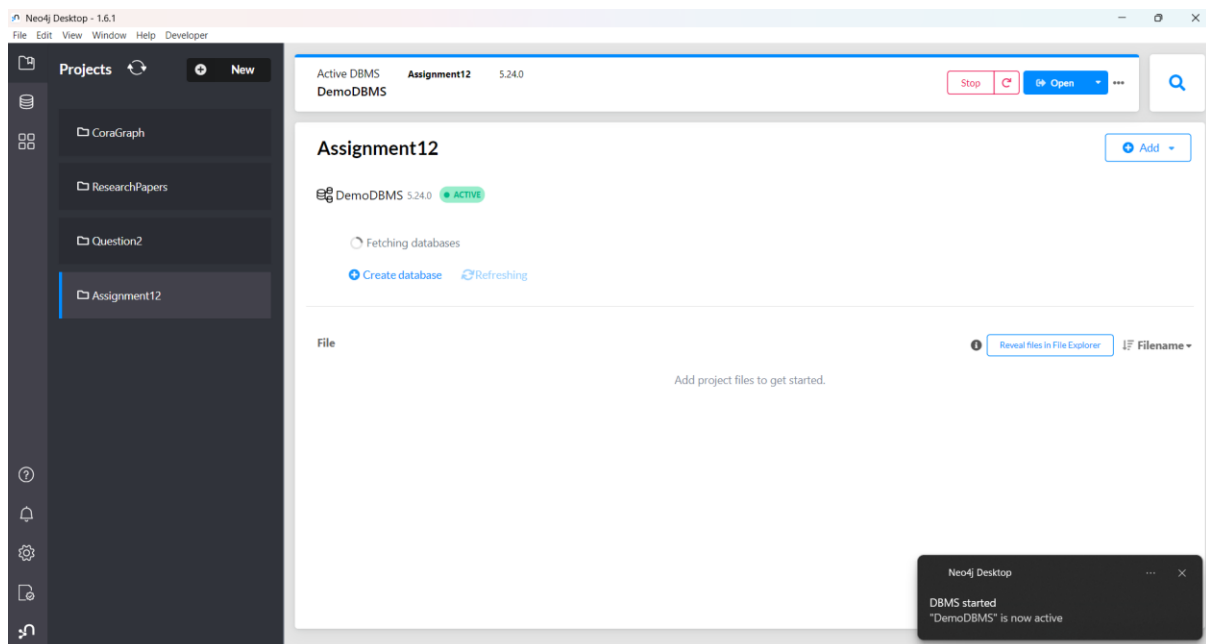
1. Write CQL (Cypher Query Language) script to add randomly 10,000 location points as follows. Assume any data.



2. Use the point() , distance() function of Neo4j to answer the queries “which things close/nearest to which other things”.
3. Demonstrate the result by firing different cypher queries (write CQL statement).

## Result:

### Creating a project and database in Neo4J



This query prints the information about neo4j system

neo4j\$

```
neo4j$ CALL dbms.info();
```

| id                                                                 | name     | creationDate               |
|--------------------------------------------------------------------|----------|----------------------------|
| "3687C0EA364B0BE7B572922FF1840E3DBAE0A0C9FC02FAF839A58254FE45C6BF" | "system" | "2025-04-15T12:55:51.745Z" |

neo4j\$ call db.schema.visualization

| nodes | relationships |
|-------|---------------|
| []    | []            |

Creating Sample Data (TrainStation → Office)

```
1 CREATE (ts:TrainStation {
2   longitude: 12.56459,
3   latitude: 55.672874,
4   city: 'Copenhagen'
5 })
6 CREATE (off:Office {
7   longitude: 12.994341,
8   latitude: 55.611784,
9   city: 'Malmo'
10 })
11 CREATE (ts)-[:TRAVEL_ROUTE]-(off);
12
```

neo4j\$ CREATE (ts:TrainStation { longitude: 12.56459, latitude: 55.672874, city: 'Copenhagen' }) CREATE (off:Office { longitu...

Added 2 labels, created 2 nodes, set 6 properties, created 1 relationship, completed after 97 ms.

Generate 10,000 Random Location Nodes

neo4j@bolt://localhost:7687/neo4j - Neo4j Browser

```
1 UNWIND range(1, 10000) AS id
2 CREATE (:Place {
3   id: id,
4   name: 'Location_' + id,
5   location: point({
6     longitude: 12.0 + rand() * 2, // roughly 12 to 14
7     latitude: 55.0 + rand() * 1 // roughly 55 to 56
8   })
9 });
10
```



## Find Nearest Places Using distance()



## Find all Places within 5km of the TrainStation



## **Conclusion**

This assignment demonstrates the application of Neo4j and its Spatial plugin for performing location-based queries on large datasets. By utilizing the `point()` and `distance()` functions, the assignment successfully illustrates how to find the nearest objects in a graph, which can be useful in various real-world applications such as geospatial search services. The integration of Neo4j with geospatial data allows for efficient querying and analysis of proximity relationships, which is crucial in applications like location-based services, geographic information systems (GIS), and route optimization.